CENTRO UNIVERSITÁRIO UNISENAI SANTA CATARINA

PROJETO APLICADO I

OTÁVIO MODOLON NETO

THIAGO NEVES GOMES

LUCAS ANTUNES SEVERINO

CONSTRUÇÃO DO MVP, PARTE 1: CLASSES EM PYTHON E CONEXÃO COM O BANCO SQLITE PARA A AUTOMAÇÃO DA EMISSÃO DE CERTIFICADOS DE CALIBRAÇÃO

Empresa parceira: Schulz S.A.

FLORIANÓPOLIS

2026

APRESENTAÇÃO

Este documento acompanha a Entrega 03 do Tema 03 e explica a primeira parte do MVP construída pelo grupo. A Entrega 01 mapeou o processo manual de emissão de certificados de calibração do laboratório de metrologia da Schulz S.A., e a Entrega 02 transformou aquele diagnóstico em um banco SQLite com as tabelas `tecnico`, `equipamento` e `medicao`. Nesta entrega o banco ganha código: classes em Python que representam as três entidades, aplicam as regras metrológicas e gravam e consultam os dados.

O código está nos arquivos do repositório indicados no fim deste documento. Aqui ficam a arquitetura escolhida, as regras de negócio implementadas, o tratamento de erros e o resultado da execução. As anomalias continuam citadas pelo código do catálogo da seção 2.2 da Entrega 01.

Conferência dos entregáveis

Item requisitado	Arquivo	Pedido no roteiro	Entregue
1. Classes de domínio e conexão SQL	`04_classes.py`	Classes POO, validação de tolerância e CRUD com `sqlite3`	4 classes, 2 exceções, INSERT, SELECT e UPDATE nas 3 tabelas
2. Script principal de demonstração	`05_main.py`	Instanciação, inserção de testes, consulta com JOIN e relatório no terminal	6 etapas, 10 medições, SELECT com 4 junções, 7 testes de erro
3. Documentação técnica do MVP	`06_mvp_parte1.md`	Padrões de POO, UML textual, tratamento de erros SQL e validação do fluxo	8 seções, diagrama Mermaid e textual, tabela de resultados

1. DO BANCO AO CÓDIGO

1.1 Ponto de partida

A Entrega 02 deixou pronto o script `03_banco.sql`, com três tabelas, quatro chaves estrangeiras e as restrições de domínio. O banco já recusava o que a planilha aceitava, como uma medição sem instrumento ou um status fora da lista. O que faltava era quem calculasse esse status e decidisse se uma medição pode ou não ser gravada. Na planilha esse papel era das fórmulas da aba Registro, que a Entrega 01 encontrou com 42 células em erro e com o limite inferior calculado ao contrário.

O MVP coloca essas decisões em classes. Cada tabela tem uma classe correspondente, e o script da AV 02 continua sendo a definição do banco: a classe de persistência o executa para criar o arquivo `certificados.db`, em vez de repetir o DDL dentro do código.

1.2 Escolha da linguagem

O grupo escolheu Python, com o módulo `sqlite3`, que faz parte da biblioteca padrão. O MVP roda com o Python 3.8 ou superior e não precisa de nenhum pacote instalado. Pesou também o que vem depois: a leitura do relatório TXT da máquina de medição, prevista para a Parte 2, é tratamento de texto, e em Python isso se resolve sem bibliotecas adicionais.

2. ARQUITETURA DE CLASSES

2.1 Classes do sistema

O arquivo `04_classes.py` tem as quatro classes pedidas no roteiro. A correspondência com o banco é direta, com os nomes de campo do dicionário da AV 02.

Classe	Tabela	Responsabilidade
`Tecnico`	`tecnico`	Pessoa ou setor que participa da calibração. Valida o nome obrigatório e a função (executante, solicitante ou signatário) e trata a matrícula como identificador.
`Equipamento`	`equipamento`	Instrumento calibrado ou padrão. Grava o código só com dígitos, valida tipo, resolução e datas, e informa se o padrão está vigente em uma data.
`Medicao`	`medicao`	Resultado de uma característica em uma execução. Calcula limites, erro e status APROVADO ou REPROVADO e bloqueia a gravação com padrão vencido.
`BancoDados`	as três	Abre a conexão `sqlite3`, liga as chaves estrangeiras, cria a estrutura e executa os comandos INSERT, SELECT e UPDATE, traduzindo os erros do SQLite.

As três classes de domínio não têm nenhum comando SQL. Elas validam os próprios dados e aplicam as regras; quem conversa com o banco é só a `BancoDados`, que recebe e devolve objetos. Essa separação faz com que uma mudança de banco no futuro, se a Schulz preferir outro gerenciador, fique restrita a uma classe.

2.2 Padrões de orientação a objetos

O encapsulamento foi feito com propriedades. Python não tem o modificador `private` do Java; a convenção é guardar o valor em um atributo com sublinhado, como `_funcao`, e expor o acesso por `@property`. O setter de cada propriedade é onde a validação acontece, e por isso um objeto inválido não chega a existir: criar um técnico com a função "estagiario" levanta erro no construtor, antes de qualquer comando ao banco.

Na classe `Medicao`, erro, status e os dois limites são propriedades sem setter. São calculados toda vez que são lidos, a partir do valor nominal, das tolerâncias e do valor medido. O status não pode ficar desatualizado em relação ao valor medido, e não existe caminho para alguém digitar um status à mão.

Cada classe de domínio tem ainda um método de fábrica, `de_linha()`, que monta o objeto a partir de uma linha lida do banco, e a `BancoDados` funciona como gerenciador de contexto: usada com `with`, fecha a conexão ao final mesmo que ocorra erro no meio. O diagrama de classes completo, em Mermaid e em versão textual, está na seção 3 do `06_mvp_parte1.md`.

3. REGRAS DE NEGÓCIO METROLÓGICAS

O roteiro pede a validação do status com base nas tolerâncias e no valor obtido. A regra implementada é: a característica está APROVADA quando o valor medido fica entre o nominal somado à tolerância inferior e o nominal somado à tolerância superior. Fora desse intervalo, está REPROVADA. A calibração inteira é reprovada se qualquer característica for reprovada.

O detalhe que importa está na soma. A tolerância inferior é gravada com sinal, negativa quando for o caso, e a planilha calculava o limite inferior como nominal menos tolerância, o que invertia a regra, como a Entrega 01 registrou. A classe calcula nominal mais tolerância. Na demonstração, a característica DISTANCIA_A_B, com nominal de 45,000 mm, tolerância de ±0,050 e valor medido de 44,961, sai aprovada; pela fórmula da planilha o limite inferior seria 45,050 e ela sairia reprovada.

As demais regras de negócio aplicadas no código estão no quadro abaixo, cada uma ligada à anomalia da Entrega 01 que ela trata.

Regra	Como funciona no código	Origem
Padrão vigente	A medição só é gravada se o padrão for do tipo padrão e tiver a próxima calibração igual ou posterior à data de execução.	A13, padrão válido até 31/08/2026 usado com cinco dias de margem
Código só com dígitos	500.011 e 500011 viram o mesmo código, 500011.	A13, código divergente entre relatório e certificado
Temperatura numérica	O texto 20,4° é aceito e gravado como 20.4.	A25, temperatura como texto
Data em formato único	Só é aceita a data AAAA-MM-DD, com ou sem hora.	A12, três datas para o mesmo evento
Ausência como nulo	Texto vazio vira nulo; não há marcador como "---" ou "Não consta".	A01, seis marcadores de ausência
Status gravado	Cada característica vai ao banco com o status calculado.	A33, conformidade não impressa
Precisão plena	O valor medido é gravado sem arredondamento; as três casas aparecem só no relatório.	RH20 da Entrega 01

Na comparação com os limites existe uma folga de um bilionésimo. Sem ela, um valor medido exatamente no limite seria reprovado por resíduo de ponto flutuante, porque 45 somado a -0,05 resulta em 44,94999... no computador. A folga é muito menor que a resolução

de 0,001 mm do instrumento e não altera nenhum resultado real; o teste de limite confirmou que 44,950 e 45,050 saem aprovados e 44,9499 e 45,0501 saem reprovados.

4. TRATAMENTO DE ERROS

O sistema separa dois tipos de erro. O `ErroValidacao` é levantado pelas classes de domínio quando um dado viola uma regra, antes de chegar ao banco. O `ErroBanco` é levantado pela `BancoDados` quando o SQLite recusa o comando. O script principal captura os dois e encerra com uma mensagem clara, sem despejar o rastro técnico na tela.

Todo comando de escrita passa por um único método, que roda o SQL dentro de uma transação. Se o comando der certo, o `sqlite3` confirma; se falhar, desfaz, e o banco não fica pela metade. O método traduz a mensagem do SQLite em uma explicação de negócio: falha de chave estrangeira vira "registro relacionado não existe no cadastro", falha de unicidade vira "registro duplicado" seguido do campo, e falha de CHECK vira "valor fora do domínio permitido".

Dois cuidados com o SQLite ficaram no código. A verificação de chave estrangeira é ligada em toda conexão, porque o SQLite não a aplica por padrão e o comando vale só para a conexão em que foi executado. E todos os comandos usam parâmetros, nunca valores concatenados no texto do SQL, o que impede injeção de SQL e problemas com aspas em nomes.

5. DEMONSTRAÇÃO E VERIFICAÇÃO

5.1 Fluxo do script principal

O `05_main.py` executa o fluxo do MVP em seis etapas: cria o banco, cadastra dois técnicos e três equipamentos, grava dez medições, monta o relatório com JOIN, registra o número do certificado com UPDATE e tenta gravar sete registros inválidos. Os dados de teste seguem o caso do certificado nº 00091/2026 da Entrega 01, com o executante, a TAG 703.1208, o setor Usinagem I e o padrão Contura G2 da Carl Zeiss. Matrículas, código do instrumento e valores medidos são fictícios.

A consulta do relatório junta a tabela `medicao` com `equipamento` duas vezes, uma para o instrumento e outra para o padrão, e com `tecnico` duas vezes, uma para o executante e outra para o solicitante. O solicitante entra por LEFT JOIN, porque a coluna aceita nulo. A saída no terminal ficou assim:

```
================================================================================
3. RELATÓRIO DE CALIBRAÇÃO (SELECT COM JOIN)
================================================================================
  Certificado nº ...: (não emitido)
  Instrumento ......: Dispositivo de controle, TAG 703.1208, código 7031208
  Padrão ...........: Máquina de medição por coordenadas Carl Zeiss, código 500011, série 502390
  Certificado padrão: 241120502390, válido até 2026-08-31
  Executante .......: José Alcides Rosa Junior (matrícula 10234)
  Solicitante ......: Usinagem I
  Data de execução .: 2026-08-13 10:15:00
  Temperatura ......: 20.4 °C

  Característica   Seq   Nominal Lim. inf. Lim. sup.    Medido     Erro  Status
  ------------------------------------------------------------------------------
  DIAMETRO_FURO_1    1    12.000    12.000    12.018    12.009    0.009  APROVADO
  DIAMETRO_FURO_1    2    12.000    12.000    12.018    12.009    0.009  APROVADO
  DISTANCIA_A_B      1    45.000    44.950    45.050    44.961   -0.039  APROVADO
  DISTANCIA_A_B      2    45.000    44.950    45.050    44.961   -0.039  APROVADO
  PLANICIDADE_REF    1     0.000     0.000     0.020     0.012    0.012  APROVADO
  PLANICIDADE_REF    2     0.000     0.000     0.020     0.013    0.013  APROVADO
  POSICAO_FURO_1     1     0.000     0.000     0.100     0.087    0.087  APROVADO
  POSICAO_FURO_1     2     0.000     0.000     0.100     0.089    0.089  APROVADO
  POSICAO_FURO_2     1     0.000     0.000     0.100     0.115    0.115  REPROVADO
  POSICAO_FURO_2     2     0.000     0.000     0.100     0.114    0.114  REPROVADO
  ------------------------------------------------------------------------------
  Medições: 10 | aprovadas: 8 | reprovadas: 2 | resultado da calibração: REPROVADO
```

A característica POSICAO_FURO_2 ficou acima da tolerância de 0,100 mm nas duas execuções, como os parâmetros de posição que a Entrega 01 encontrou fora da tolerância sem nenhuma coluna de conformidade. Aqui o status aparece em cada linha e a calibração sai reprovada no resumo.

5.2 Resultado da verificação

Nº	Teste	Esperado	Resultado obtido
V01	Criar o banco pelo `03_banco.sql`	3 tabelas	3 tabelas criadas
V02	Cadastrar técnicos e equipamentos	2 e 3 registros	Gravados; 500.011 gravado como 500011
V03	Gravar as medições com status calculado	10 linhas, 2 reprovadas	10 linhas, POSICAO_FURO_2 reprovada nas duas execuções
V04	Consultar o relatório com JOIN	Cabeçalho e 10 linhas	Exibido, resultado geral REPROVADO
V05	Registrar o certificado 00091/2026 (UPDATE)	10 linhas alteradas	10 linhas, conferidas pelo SELECT
V06	Medição com padrão vencido	Recusa	ErroValidacao: padrão 500020 vencido em 2026-06-30
V07	Mesma característica duas vezes na mesma execução	Recusa	ErroBanco: registro duplicado na chave única composta
V08	Medição com técnico inexistente	Recusa	ErroBanco: chave estrangeira
V09	Equipamento com código já cadastrado	Recusa	ErroBanco: registro duplicado em equipamento.codigo
V10	Técnico com função fora da lista	Recusa	ErroValidacao, antes do banco

V11	Tolerância inferior maior que a superior	Recusa	ErroValidacao, antes do banco
V12	Data no formato 13/08/2026	Recusa	ErroValidacao, antes do banco

Os doze resultados ficaram como o esperado. O V06 é o que mais se aproxima do risco real do processo: com a planilha, o padrão vencido só era percebido se alguém conferisse a data a olho; com o MVP, a gravação é bloqueada e a mensagem diz qual padrão venceu e quando.

6. AJUSTE NA AV 02 E PRÓXIMOS PASSOS

Para atender ao roteiro desta entrega, o domínio da coluna `status` no `03_banco.sql` passou de "conforme" e "nao conforme" para APROVADO e REPROVADO, e o `02_dicionario.md` foi atualizado na mesma linha. Foi a única mudança na estrutura do banco. Os cabeçalhos dos arquivos também tiveram a grafia do nome de Lucas Antunes Severino corrigida.

Esta parte do MVP não lê o relatório TXT da máquina de medição nem gera o certificado. Os resultados entram pelo script de demonstração. Ficam para a Parte 2 a rotina de leitura do relatório com as regras de saneamento da Entrega 01, a separação da entidade de calibração prevista na Entrega 02, para que número do certificado, data e temperatura deixem de se repetir em cada linha, e a geração do certificado a partir do relatório que hoje sai no terminal.

7. ARQUIVOS ENTREGUES

Arquivo	Conteúdo
`04_classes.py`	Classes Tecnico, Equipamento, Medicao e BancoDados, e as exceções do sistema
`05_main.py`	Script de demonstração do MVP
`06_mvp_parte1.md`	Documentação técnica do MVP, Parte 1
`03_banco.sql`	Script DDL da AV 02, com o domínio do status ajustado
`Entrega_03_PA_I.pdf`	Este documento

Para executar, na pasta do repositório: `python 05_main.py`

Repositório: https://github.com/ModolOff/Projeto-Aplicado-I---AV2---Modelagem-e-Estruturacao-do-Banco-de-Dados